



uv

Der moderne Python-Paketmanager und -Builder

Daniel Senften

Version 1.0, 2025-10-14

Inhaltsverzeichnis

1. Einführung	2
1.1. Warum uv?	2
2. Installation	3
2.1. Systemvoraussetzungen	3
2.2. Installation über Homebrew (macOS) - Empfohlen	3
2.3. Installation über pip	3
2.4. Installation über curl (Linux/macOS)	3
2.5. Installation über pipx (alle Plattformen)	3
2.6. Installation unter Windows	4
2.7. Überprüfung der Installation	4
3. Python-Versionen verwalten	5
3.1. Python-Version anzeigen	5
3.2. Spezifische Python-Version installieren	5
3.3. Python-Version für Projekt festlegen	5
4. Neue Projekte initialisieren	6
4.1. Neues Projekt erstellen	6
4.2. Projekt mit Abhängigkeiten hinzufügen	6
4.3. Projekt synchronisieren	6
4.4. Code ausführen	6
5. Grundlegende Verwendung (Legacy-Modus)	7
5.1. Pakete installieren	7
5.2. Virtuelle Umgebungen	7
5.3. Mit requirements.txt	7
5.4. Mit pyproject.toml	7
6. Vergleich: uv vs pip vs poetry	8
6.1. Geschwindigkeit	8
6.2. Funktionen	8
6.3. Kompatibilität	8
7. Fortgeschrittene Nutzung	9
7.1. Lock-Dateien erstellen	9
7.2. Abhängigkeiten aktualisieren	9
7.3. Entwicklungsumgebung einrichten	9
8. Zusammenarbeit der Tools	10
8.1. Können pip, poetry und uv zusammen verwendet werden?	10
9. Best Practices	12
10. Fehlerbehebung	13
10.1. Häufige Probleme	13
11. Cheat Sheet - Wichtigste Befehle	14

11.1. Installation und Setup	14
11.2. Neue Projekte	14
11.3. Legacy-Modus (pip-Ersatz)	14
11.4. Nützliche Optionen	15
12. Fazit	16
13. Nächste Schritte	17
14. Glossar	18

`<a href="https://github.com/astral-sh/uv">[uv]</a> | <em>https://img.shields.io/badge/uv-FFD43B?logo=python&logoColor=blue&style=for-the-badge</em>`

1. Einführung

`uv` ist ein schneller Python-Paketmanager und -Builder, der in Rust geschrieben wurde. Er wurde von den Machern von `ruff` entwickelt und bietet eine moderne Alternative zu `pip` und `poetry`. Mit `uv` können Sie Python-Pakete schneller installieren, virtuelle Umgebungen verwalten und Python-Projekte bauen und veröffentlichen.

In diesem Tutorial lernen Sie

- Installation von `uv` auf verschiedenen Systemen
- Initialisierung neuer Python-Projekte mit `uv`
- Python-Versionen verwalten mit `uv`
- Moderne Paketmanagement-Workflows
- Grundlegende Befehle für das Paketmanagement
- Vergleich von `uv` mit `pip` und `poetry`
- Verwendung von `uv` in bestehenden Projekten
- Erstellen und Verwalten virtueller Umgebungen
- Builden und Veröffentlichen von Paketen

1.1. Warum `uv`?

- **Schneller** als herkömmliche Python-Paketmanager
- **Kompilierung in Rust** für bessere Leistung
- **Kompakte Binärdatei** ohne Python-Abhängigkeiten
- **Drop-in-Ersatz** für `pip` und `pip-tools`
- **Schnellere Installation** von Paketen durch parallele Downloads
- **Bessere Fehlermeldungen** und Benutzererfahrung
- **Unterstützung** für `pyproject.toml` und `requirements.txt`

2. Installation

2.1. Systemvoraussetzungen

- Python 3.8 oder höher
- Linux, macOS oder Windows
- Für optimale Leistung: Python 3.9+

2.2. Installation über Homebrew (macOS) - Empfohlen

Für macOS-Benutzer ist Homebrew die bequemste Installationsmethode:

```
brew install uv
```



Nach der Installation mit Homebrew müssen Sie möglicherweise den PATH konfigurieren. Führen Sie `uv tool update-shell` aus, um dies automatisch zu erledigen.

2.3. Installation über pip

Die einfachste plattformübergreifende Möglichkeit:

```
pip install uv
```

2.4. Installation über curl (Linux/macOS)

Direkte Installation vom offiziellen Installer:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Diese Methode konfiguriert automatisch den PATH in `~/.zshenv`.

2.5. Installation über pipx (alle Plattformen)

Für isolierte Installation:

```
pipx install uv
```

2.6. Installation unter Windows

2.6.1. PowerShell (empfohlen)

```
irm https://astral.sh/uv/install.ps1 | iex
```

2.6.2. Über pip

```
pip install uv
```

2.6.3. Über winget

```
winget install astral-sh.uv
```

2.7. Überprüfung der Installation

```
uv --version
```

3. Python-Versionen verwalten

uv kann Python-Versionen automatisch installieren und verwalten:

3.1. Python-Version anzeigen

```
uv python list
```

3.2. Spezifische Python-Version installieren

```
uv python install 3.12  
uv python install 3.11.5
```

3.3. Python-Version für Projekt festlegen

```
uv python pin 3.12
```


4. Neue Projekte initialisieren

4.1. Neues Projekt erstellen

```
# Neues Projekt im aktuellen Verzeichnis
uv init

# Neues Projekt in neuem Verzeichnis
uv init mein-projekt
cd mein-projekt
```

Dies erstellt: - `pyproject.toml` - Projektkonfiguration - `README.md` - Projektdokumentation - `src/` - Source-Code-Verzeichnis - `.python-version` - Gepinnte Python-Version

4.2. Projekt mit Abhängigkeiten hinzufügen

```
# Abhängigkeit hinzufügen
uv add requests pandas

# Entwicklungsabhängigkeiten hinzufügen
uv add pytest black --dev
```

4.3. Projekt synchronisieren

```
# Alle Abhängigkeiten installieren
uv sync

# Ohne Entwicklungsabhängigkeiten
uv sync --no-dev
```

4.4. Code ausführen

```
# Python-Skript mit uv ausführen
uv run python main.py

# Direkt ein Modul ausführen
uv run pytest
```

5. Grundlegende Verwendung (Legacy-Modus)

5.1. Pakete installieren

Installieren von Paketen ähnlich wie mit `pip`:

```
uv pip install requests pytest
```

5.2. Virtuelle Umgebungen

Erstellen einer neuen virtuellen Umgebung:

```
uv venv .venv
```

Aktivieren der virtuellen Umgebung:

```
# Auf Unix/macOS
source .venv/bin/activate

# Auf Windows
.venv\Scripts\activate
```

5.3. Mit requirements.txt

```
uv pip install -r requirements.txt
```

5.4. Mit pyproject.toml

```
uv pip install -e .
```

6. Vergleich: uv vs pip vs poetry

6.1. Geschwindigkeit

- **uv** ist deutlich schneller als **pip** und **poetry**
- Parallele Paketinstallation
- Effizienteres Caching

6.2. Funktionen

Feature	uv	pip	poetry
Parallele Installation	☑	☐	☐
Lock-Dateien	☐	☐	☐
Abhängigkeitsauflösung	☐	Eingeschränkt	☐
Virtuelle Umgebungen	☐	☐	☐
Build-System	☐	☐	☐
Publishing	Eingeschränkt	☐	☐

6.3. Kompatibilität

- **uv** ist weitgehend kompatibel mit **pip** und **pip-tools**
- Unterstützt **requirements.txt** und **pyproject.toml**
- Kann in bestehenden Projekten parallel zu **pip** verwendet werden

7. Fortgeschrittene Nutzung

7.1. Lock-Dateien erstellen

```
uv pip compile requirements.in -o requirements.txt
```

7.2. Abhängigkeiten aktualisieren

```
uv pip install --upgrade-package package_name
```

7.3. Entwicklungsumgebung einrichten

```
uv pip install -e ".[dev]"
```

8. Zusammenarbeit der Tools

8.1. Können pip, poetry und uv zusammen verwendet werden?

Grundsätzlich ist es technisch möglich, **pip**, **poetry** und **uv** im selben Projekt zu verwenden, aber es gibt wichtige Überlegungen:

1. Nicht gleichzeitig verwenden

- Vermeiden Sie es, die Tools parallel für das gleiche virtuelle Environment zu verwenden
- Jedes Tool verwaltet Abhängigkeiten auf seine eigene Weise
- Parallele Nutzung kann zu Inkonsistenzen führen

2. Empfohlener Ansatz

- Wählen Sie ein Haupttool für das Dependency-Management
- Nutzen Sie die anderen Tools nur für spezifische Aufgaben
- Dokumentieren Sie die gewählte Vorgehensweise im Projekt

3. Typische Kombinationen

- **Poetry + uv** (empfohlen):
 - Verwenden Sie **poetry** für die Abhängigkeitsverwaltung
 - Nutzen Sie **uv** als schnellen **pip**-Ersatz für Installationen
 - Beispiel:

```
# Mit poetry Abhängigkeiten definieren
poetry add package

# Mit uv schnell installieren
uv pip install -r requirements.txt
```

- **Nur uv** (für einfache Projekte):
 - Nutzen Sie **uv** als vollständigen Ersatz für **pip**
 - Ideal für einfache Projekte ohne komplexe Abhängigkeiten +
- **Poetry + pip** (traditionell):
 - Verwenden Sie **poetry** für das Management
 - Nutzen Sie **pip** für spezielle Fälle

4. Best Practices

- Halten Sie sich an ein konsistentes Toolset pro Projekt
- Dokumentieren Sie die verwendeten Tools in der **README.md**
- Nutzen Sie **.gitignore** für tool-spezifische Dateien

- Vermeiden Sie manuelle Änderungen an `pyproject.toml` oder `poetry.lock`
- Verwenden Sie in CI/CD-Pipelines das gleiche Tool wie lokal

5. Wann welches Tool verwenden?

Anwendungsfall	Empfohlenes Tool	Alternative
Neue Projekte mit Abhängigkeiten	<code>poetry</code>	<code>uv</code>
Einfache Skripte	<code>uv</code>	<code>pip</code>
CI/CD-Pipelines	<code>uv</code> (schneller)	<code>pip</code>
Komplexe Abhängigkeiten	<code>poetry</code>	
Legacy-Projekte	<code>pip</code>	

6. Warnhinweise

- Vermeiden Sie es, `pip install` in einer mit `poetry` verwalteten Umgebung zu verwenden
- Seien Sie vorsichtig bei der manuellen Bearbeitung von `pyproject.toml`
- Führen Sie nach Tool-Wechsel immer einen sauberen Neustart durch

9. Best Practices

1. Verwenden Sie **uv** für schnelle Installationen in CI/CD-Pipelines
2. Kombinieren Sie mit **poetry** für besseres Dependency Management
3. Nutzen Sie **.gitignore** für **.venv/** und **pycache/**
4. Dokumentieren Sie die Installation von **uv** in Ihrer README
5. Aktualisieren Sie regelmässig **uv** selbst

10. Fehlerbehebung

10.1. Häufige Probleme

1. Berechtigungsprobleme

- Verwenden Sie `--user` oder virtuelle Umgebungen

2. Inkompatible Pakete

- Versuchen Sie es mit `--no-deps` und installieren Sie Abhängigkeiten manuell

3. Netzwerkprobleme

- Überprüfen Sie Ihre Internetverbindung
- Verwenden Sie `--timeout` für langsame Verbindungen

11. Cheat Sheet - Wichtigste Befehle

11.1. Installation und Setup

```
# Installation (macOS mit Homebrew)
brew install uv

# Installation (andere Plattformen)
pip install uv
curl -LsSf https://astral.sh/uv/install.sh | sh # Linux/macOS
pipx install uv                                # Isolierte Installation

# PATH konfigurieren (falls nötig)
uv tool update-shell

# Version prüfen
uv --version

# Python-Versionen verwalten
uv python list
uv python install 3.12
uv python pin 3.12
```

11.2. Neue Projekte

```
# Neues Projekt erstellen
uv init
uv init mein-projekt

# Abhängigkeiten hinzufügen
uv add requests pandas
uv add pytest --dev

# Projekt synchronisieren
uv sync
uv sync --no-dev

# Code ausführen
uv run python main.py
uv run pytest
```

11.3. Legacy-Modus (pip-Ersatz)

```
# Virtuelle Umgebung erstellen
uv venv .venv
```

```
# Aktivieren (Unix/macOS)
source .venv/bin/activate

# Aktivieren (Windows)
.venv\Scripts\activate

# Pakete installieren
uv pip install requests pytest
uv pip install -r requirements.txt
uv pip install -e .

# Abhängigkeiten kompilieren
uv pip compile requirements.in -o requirements.txt
```

11.4. Nützliche Optionen

```
# Upgrade einzelner Pakete
uv pip install --upgrade-package package_name

# Entwicklungsabhängigkeiten
uv pip install -e ".[dev]"

# Timeout für langsame Verbindungen
uv pip install --timeout 120 package_name

# Benutzerweite Installation
uv pip install --user package_name
```

12. Fazit

`uv` ist eine leistungsstarke Ergänzung zum Python-Ökosystem, das die Geschwindigkeit der Paketverwaltung erheblich verbessert. Während es noch nicht alle Funktionen von `poetry` abdeckt, ist es eine ausgezeichnete Wahl für schnelle Installationen und CI/CD-Pipelines.

13. Nächste Schritte

- [Offizielle Dokumentation](#)
- [Benchmarks](#)
- [Beitragen zum Projekt](#)

14. Glossar

- **uv**: Ein schneller Python-Paketmanager und -Builder
- **pip**: Der Standard-Paketmanager für Python
- **poetry**: Ein Tool für Abhängigkeitsverwaltung und Packaging
- **Virtuelle Umgebung**: Isolierte Python-Umgebung für Projekte
- **Lock-Datei**: Datei mit exakten Versionen aller Abhängigkeiten